

[DC] Toward Self-Adaptive 3D User Interfaces

Erin Argo Student Member, IEEE, *

ABSTRACT

3D User Interfaces for Augmented and Virtual Reality operate in highly dynamic, embodied, and context-dependent environments, where fixed interaction rules and static layouts are insufficient. While many XR systems incorporate runtime adaptation or heuristic optimization, these approaches differ fundamentally from true closed-loop adaptation as defined in Adaptive Software research. This work lays foundational ground for defining *Adaptive 3D User Interfaces* by grounding interface adaptation in established self-adaptive systems principles. We identify core interface properties—form, content, placement, and interaction—as primary candidates for adaptation. We present a controlled, ontology-driven adaptive interface framework that enables reproducible evaluation of adaptive behavior. Finally, we outline an experimental protocol for assessing user performance, preference, and trust under adaptive interface conditions.

Index Terms: Self-Adaptive Systems, Adaptive User Interfaces, 3D User Interfaces, Extended Reality.

1 INTRODUCTION

In Extended Reality, the point-and-click paradigms that rule 2D interfaces fall short of end-user expectations. Users in Extended Reality Applications expect interactions to behave more like physical manipulation. To create an Extended Reality system that remains usable, intelligible, and effective across diverse real-world contexts, interface design must move beyond static layouts and fixed interaction mappings. This will elevate user experience as end-users expect interacting with the XR environment to be physical manipulation rather than 2D interactions [5].

This shift toward physically grounded interaction fundamentally alters the design space of XR interfaces. XR environments are inherently dynamic, embodied, and situated within unbounded real-world settings. As a result, fixed interaction rules and static layouts are insufficient to accommodate the variability introduced by user movement, environmental conditions, and task context. To address this, contemporary XR systems increasingly incorporate adaptive or learning-based mechanisms that autonomously adjust interface behavior. As adaptive or learning based elements become more common in XR systems today, it is important to also have an effective means of evaluating them [8]. Adaptive Software, surveyed and formally defined by Salehie and Tahvildari [8], is the set of principles that make a system adaptive that allows any open loop interaction to become a closed loop interaction. Even though interaction for 3D User Interfaces is often implemented using runtime adaptation and heuristic optimization logic. These approaches differ fundamentally from adaptive programming.

This sort of runtime adaptation for 3D user interfaces is commonly used to optimize interaction flow, manage state transitions, or respond to user input based on predefined conditions. While such systems may operate in a closed loop—responding to user actions and updating system state—they typically assume a fixed problem

formulation. Adaptive Software can contribute to this ecosystem by providing a “system decider” that acts in *autonomy* for any given context or interaction. This gives the ability to turn any heuristically static software into an adaptive one.

This work aims to lay the foundation for defining Adaptive 3D User Interfaces, identify properties of interfaces that stand to benefit most from adaptation, evaluate how different adaptation strategies affect user performance and preference, improve trust for end-users by using explainable AI techniques, observe whether adaptation is appropriate for high stakes fields, set standards for consistency and verifiability of adapted interfaces, and explore techniques to allow direct user agency over adaptive technologies – especially in noisy contexts where interpretability and learnability of adaptive interfaces may be low.

2 RELATED WORK

Adaptive Software describes the set of qualifications required for a fully autonomous system. Formalized by Salehie and Tahvildari [8], there are four primary properties that define Self-Adaptive systems; it is the Self-Configuring and Self-Optimizing principles that this work stands to gain from the most. Self-Optimizing software adjusts its own configuration and parameters in response to system usage while Self-Configuring software dynamically composes and decomposes software entities.

3D User Interfaces are those that are used by Mixed Reality systems. 3DUIs can be described as a set of properties that are composed to a single uniform software entity. These properties are their **form** [4], their **content** [6], their **placement** [2, 6, 4], and their **interaction** [3, 4]. Specifically, this proposal seeks to adapt all of these properties in an effort to create a Self-Optimizing and Self-Configuring interface that dynamically and autonomously adapts to a user’s environment at any given time without needing to pre-program for that scenario. There are many different ways to adapt these interface properties as discovered in the literature but there are two overall categories which can be ascribed to these methods – **traditional** and **AI/ML**. Traditional adaptive methods are those that adjust the above properties without the use of Machine Learning. Examples of such methods are Combinatorial Optimization and Rule Based Logic systems or even an Ontology Reasoner. The primary issue with such methods is that they are still not *truly* adaptive. It is easier to map out every possibility and parameter that a given system *should* encounter but it still lacks the ability to handle every possibility that a system *can* encounter. In this way, Machine Learning is a very powerful tool that may be able to meet this end.

3 CREATING ADAPTIVE INTERFACES

In preliminary work, we implemented a method, proposed by Stefanidi et al. [10], by which interfaces may be adapted. Using an ontology-based forward chain reasoner and combinatorial optimizer, the properties of an interface may be adapted based on a set of rules. As stated in [section 2](#), this is particularly problematic when faced with parameters the system is not defined for.

3.1 Traditional Method of Adaptation

The setup is as follows: a context-aware module collects data from the simulation and passes it into the forward-chain ontology reasoner. This reasoner takes a set of pre-defined rules and the context gathered from the scene to make inferences about relationships [1].

*E. Argo is with the Department of Computer & Cyber Sciences, Augusta University, Augusta, Georgia, USA

The inferences made by the reasoner can then be used to compose interfaces.

The parsed data directly adjusts relationship-weights for objects. To compose the interface, an optimizer uses the parsed data to search for keyword matches for content in the interface sub-elements and then rank them in terms of the weights. This is done via combinatorial optimization. The interface is composed of “content suggestions” of all of the relationship-value-keyword pairs that are closest to the user (in this case, with a max of three items.) To allow user freedom and control, access to the main menu is always available and present.

3.2 Future Implementations

This heuristically-grounded setting allows for comprehensive verification of the feasibility of adaptive systems. More importantly, this controllability gives the opportunity to set rigorous standards by which to test adaptive interfaces as well as explore best practices prior to working with systems that can handle more context.

The next phase in this project will create a novel method by which interfaces may be adapted. This may be done through a combination of prompting an LLM with context obtained from a scene and, through some mathematical transformations on the output, obtain a series of interface elements that may be composed into a single interface. The data structures involved in this setup are still being determined but it is likely to involve a setup similar to previous work that has explored adaptive software using LLM reasoning with Knowledge Graphs and Monte-Carlo Tree Searches [7, 9]. In this way, not only will the final adaptation be truly autonomous, but it will be capable of adapting more than just one interface property at any given time.

4 EVALUATING USER EXPERIENCE WITH ADAPTIVE INTERFACES

This proposal has already completed work in designing an experimental protocol to evaluate the performance and efficacy of Adaptive Interfaces. Specifically, in an effort to answer the following research questions:

- Compared to a baseline interface with no adaptation; how does a rule based system or machine-learning based adaptations perform when considering time, errors, and workload under defined context changes such as alterations to environment, task, or user intent?
- Do specific agency mechanisms increase trust and perceived control without reducing performance (as in R1) or increasing workload?
- With a fixed adaptation technique, which interface properties produce the best tradeoff in performance (as in R1) and user preference?

To meet this end, an experimental trial has been devised: In a low visibility environment and given free use of an interface, a participant must complete a series of tasks to “clear” rooms. These tasks are items such as “Find all objects of X relationship” so as to evaluate the semantic connections the interface operates on. The user must search the randomly generated rooms within this low visibility environment to find each object and select their icon from the given interface to clear all the rooms and complete the trial. Specifically, the data being collected from this trial is menu navigation time, task completion time, selection errors, and user preference. By using the controlled adaptive interface from section 3, this method of evaluation may be simulated and tested to set standards before more autonomous solutions are explored and allows for comparisons to be drawn from one method to the next. There are issues with this setup in attempting to evaluate users solving every-day tasks. Specifically

in mitigating bias while allowing user-freedom. The proposed task does not do enough to emulate real-life scenarios and give an environment that is truly representative of what end-users might encounter.

5 DISCUSSION AND FUTURE WORK

This work lays foundational ground for defining, implementing, and evaluating *Adaptive 3D User Interfaces* through the lens of Adaptive Software principles. We identify core interface properties—form, content, placement, and interaction—as primary candidates for adaptation. We first plan to expand upon methods of adaptivity (such as rule-based or machine-learning based) through adapting interface content and placement before laying groundwork and principles for adapting other interface properties such as interaction. Future work involves exploring explainable autonomous systems and user trust.

The contributions of this work provide a principled definition of Adaptive 3D User Interfaces grounded in established self-adaptive systems theory, proposes and implements a controlled adaptive interface framework, and outlines an experimental protocol designed to systematically measure user performance, preference, and trust.

To strengthen the proposed research evaluation protocols must be expanded to better approximate real-world task complexity while preserving experimental control. Addressing these considerations will be essential for transitioning adaptive 3D user interfaces from controlled experimental settings to reliable real-world deployment.

ACKNOWLEDGMENTS

The author wishes to thank Drs. Jerônimo Grandi and Regis Kopper.

REFERENCES

- [1] A. Brännström and J. C. Nieves. UnityIIS: Interactive Intelligent Systems in Unity, 12 2021. 1
- [2] Y. F. Cheng, C. Gebhardt, and C. Holz. Interactionadapt: Interaction-driven workspace adaptation for situated virtual reality environments. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–14, 2023. 1
- [3] W. Gong, S. Santosa, T. Grossman, M. Glueck, D. Clarke, and F. Lai. Affordance-based and user-defined gestures for spatial tangible interaction. In *Proceedings of the 2023 ACM Designing Interactive Systems Conference*, pp. 1500–1514, 2023. 1
- [4] F. He, X. Hu, X. Qian, Z. Zhu, and K. Ramani. Adaptui: Adaptation of geometric-feature-based tangible user interfaces in augmented reality. *Proceedings of the ACM on Human-Computer Interaction*, 8(ISS):44–69, 2024. 1
- [5] J. J. LaViola Jr, E. Kruijff, R. P. McMahan, D. Bowman, and I. P. Poupyrev. *3D user interfaces: theory and practice*. Addison-Wesley Professional, 2017. 1
- [6] F. Lu and Y. Xu. Exploring spatial ui transition mechanisms with head-worn augmented reality. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, pp. 1–16, 2022. 1
- [7] E. Markowitz, A. Ramakrishna, J. Dhamala, N. Mehrabi, C. Peris, R. Gupta, K.-W. Chang, and A. Galstyan. Tree-of-traversals: A zero-shot reasoning algorithm for augmenting black-box language models with knowledge graphs. *arXiv preprint arXiv:2407.21358*, 2024. 2
- [8] M. Salehie and L. Tahvildari. Self-adaptive software: Landscape and research challenges. *ACM transactions on autonomous and adaptive systems (TAAS)*, 4(2):1–42, 2009. 1
- [9] X. Song, S. Zhang, and T. Yu. Rekg-mcts: Reinforcing llm reasoning on knowledge graphs via training-free monte carlo tree search. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 9288–9306, 2025. 2
- [10] Z. Stefanidi, G. Margetis, S. Ntoa, and G. Papagiannakis. Real-time adaptation of context-aware intelligent user interfaces, for enhanced situational awareness. *IEEE Access*, 10:23367–23393, 2022. 1